

Goals

Goals: model can predict new edges in a graph over time

- This will likely involved merging a GCN (likely attentions based) with a recurrent model, likely a LSTM
- Advanced: models can predict new nodes
 - generate node (from LSTM hidden state or dedicated sub-network that turns subgraph into LLM input)
 - * It can also just select nodes from candidate states we init the network with
 - Could actually be better because user can define entites and descriptions
 - * need generative text models (fine tuned LLMs) for this
 - generate edges using link prediction
 - * top k from sample triples sampled from subgraph and edge types

Notes

Notes: Model will be created with global node pool. Some nodes will be unconnected. This is ok.

Representation

First need to consider how to represent the text in edges and nodes

Edge embeddings

Edge embeddings

- need to be a limited subset that
- embeddings will be learned by the model
- Ex: born, died, friends, enemies, killed_by, etc

Node text

Then how to rep node text:

2 options

- One-hot encoding (in hindsight I dont think this is plausible. To little info)
 - Pros

- * Easy decode rule (take softmax and sample probabilities)
- * Super easy to implement (nn.Embedding layer)
- Cons
 - * Less Representative and would probably worsen the model
 - * Can get big fast because of the copious amount of proper nouns in the dataset
- Actual Embedding layer
 - Pros
 - * Much more representative
 - * Size is more manageable
 - Cons
 - * Much more difficult to implement
 - imported models will probably struggle with this task because of all the proper nouns, but adding our own embedding layers will make the model more expensive and difficult
 - * Decoding is much more difficult (only actually needed for node generation (advanced))
 - We would probably have to extract the entire vocab of whatever model we end up using and do a dot product search for closest word
 - further complicated by the fact tokens wont be full words, and edge predictions will likely have to be one shot.
 - Or we could go with the above idea to just generate a graph embedding and throw it into a fine tuned LLM

Text embedding could also be merged with structural embedding in some way, but the structure will probably be learned via convolutions so its probably ok.

With embedding taken care of each node will be represented by a embedding and we have the starting structure for the graph.

GNN Update

Update Rule:

$$h_i^{(t+1)} = \text{Non-Linearity} \left(\sum_j a_{ij} \cdot m_{ij} \right)$$

where a_{ij} is a attention score and m is message (linear combination of messages)

- attention scores are calculated by

$$a_{ij} = \text{Non-Linearity}(a(z_i, z_j, \text{edge}_{ij}))$$

where a_{ij} is attention score, a is learned attention function and

$$z_i = W_n \cdot h_i$$

Do it only over neighbors

- gotta do softmax to get a_{ij}

$$a_{ij} = \frac{\exp(\text{Non-Linearity}(a(z_i, z_j, \text{edge}_{ij})))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{Non-Linearity}(a(z_i, z_k, \text{edge}_{ik})))}$$

- messages (need a way to synthesize edges and nodes)

$$m_{ij} = W_h \cdot h_j^{(t)} + W_r \cdot \text{edge}_{ij}$$

Multi-head attention

- Multi head attentions should be considered after single head is finished. Basically the same thing but project down and concat

- dim has to be divisible by number of heads
- Same set of the above equations for each head (so head has separate, W_h , W_r , etc scaled down to project into $\text{dim}/\#$ heads)
- at end concat output of each head for full

Further considerations

Further considerations

- since we are using sum should probably use layer norms to keep things from exploding and maybe residuals
- Something like this

$$\begin{aligned} h^{(t+1)} &= \text{GNNLayer}(h^{(t)}) \\ h^{(t+1)} &= \text{LayerNorm}(h^{(t+1)} + h^{(t)}) \end{aligned}$$

- Note: if using multi head attention need to project $h^{(t)}$ down
- can also add a feedforward network after attention like GNNs

$$h^{(t+1)} = \text{LayerNorm}(h^{(t+1)} + \text{MLP}(h^{(t+1)}))$$

- dropouts could also be useful for better generalization

Running this for a bit will give us rich contextual embeddings for each node, which we can then use for link prediction and node gen

Link prediction

Link prediction

- consider possible edges between node i and node j
- Feed (i, j, edge) into a basic MLP and get a score
 - condition on LSTM hidden state for more context
 - $\text{MLP}(i, j, \text{edge}, \text{lstm_hidden})$
- has to be over a subset of nodes and edges or it become unfeasible
 - node sampling strategies (candidates j for each i)
 - * random (probably a bad idea but a bit of noise isnt bad)
 - * neighbors (selection of nodes of depth 1–3 away from i)
 - * embedding similarity (dot product with node embedding matrix (could be expensive))
 - edge sampling
 - * total random (probably a bad idea but a bit of noise isnt bad)
 - * build counts of edges in neighbors and sample probabilistically from it based on count (with noise)
 - count also be based off how good edge is, so $\text{MLP}(i, j, \text{edge})$ but that might be to expensive
 - * TODO: need more possibilities
- batch all triplets into single tensor and pass to scoring MLP
- Apply sigmoid to get probs
- possible link selection strategies (should it be allowed to create more than one link per timestep?)
 - anything that passes a certain threshold (variable multi link)
 - top k (fixed multi-link)
 - apply softmax and sample probabilistically (fixed multi link)
- add to graph tables

Node gen (Advanced)

Node gen (Advanced)

- Note: this would run before the GNN
- sample from nodes in candidate pool (unconnected ones in global node pool)

- true node get (probably too much)
 - * generate new node on LSTM hidden state
 - probably via `finetuned_LLM(lstm_hidden)`
 - * embed text to create new node embedding
- run link prediction with new node
 - this is hard because we have nothing to base this off
 - could randomly sample
 - or use type constraints if we implement typing
- add to graph tables

LSTM update

After Link prediction update LSTM hidden state

- $x = \text{ReadOut}(H)$ # convert graph to vector for LSTM
 - probably attention pooling
 - * compute attention score and readout becomes a linear combination of node embedding weighed by attention scores
 - * can also be conditioned on LSTM hidden state
- Standard LSTM update using x

Data considerations

Data considerations:

- structure has to be suitable for pytorch
 - this is more of a code thing. We're gonna have to figure this out
- a strict typing system would provide a lot of structure that could be exploited
 - unless the data already has it we would probably need a llm pipeline to type everything
 - built into wikidata?
- constrained number of relationships
 - this could also be a pain if not in dataset